

C++&Qt Day3 hw1 보고서

알고리즘의 이해

휴머노이드 수습단원 이주원

목차

1. 알고리즘이란?
 - 1.1 시간복잡도
 - 1.2 공간복잡도
2. 다익스트라 알고리즘
3. A* 알고리즘

1. 알고리즘이란?

- 알고리즘이란 문제를 해결하기 위한 절차나 방법을 의미하는 단어로 넓은 범위에서 사용되거나 행동을 하기 위해서 만들어진 명령들의 유한 집합

컴퓨터과학에서 말하는 알고리즘은 보통 반복되는 문제를 풀기 위한 작은 진행절차를 의미

- 알고리즘의 조건

1. 입력: 알고리즘은 0 또는 그 이상의 외부에서 제공된 자료가 존재해야 한다.
2. 출력: 알고리즘은 최소 1개 이상의 결과를 가져야 한다.
3. 명확성: 알고리즘의 각 단계는 명확하여 애매함이 없어야 한다.
(난수를 사용하는 절차 또한 난수의 확률 분포가 가져야 할 특성이 명백하다면 알고리즘)
4. 유한성: 알고리즘은 단계들을 유한한 횟수로 거친 후 문제를 해결하고 종료해야 한다.
5. 효과성: 알고리즘의 모든 연산들은 사람이 종이와 연필을 이용하여 유한한 시간 안에 정확하게 수행할 수 있을 정도로 충분히 단순해야 한다.

알고리즘은 입력에 따라 명령을 명확하게 실행하고, 효과적으로 입력에 따른 결과를 도출

명령의 애매함이 있고 유한한 시간 안에 끝난다는 보장이 없다면 매서드(Method)

- 알고리즘이 위 조건을 모두 만족한다고 해서 '효과적으로' 풀어낸다고 할 수는 없음.
(유한한 시간이 몇 일 혹은 몇 년이 될 수도 있기 때문)

따라서 컴퓨터에서는 시간과 메모리라는 두 자원을 얼마나 소모하는지가 효율성의 중점



<알과리즈미> 알고리즘 단어의 유래된 이름

1.1 시간복잡도(Time Complexity)

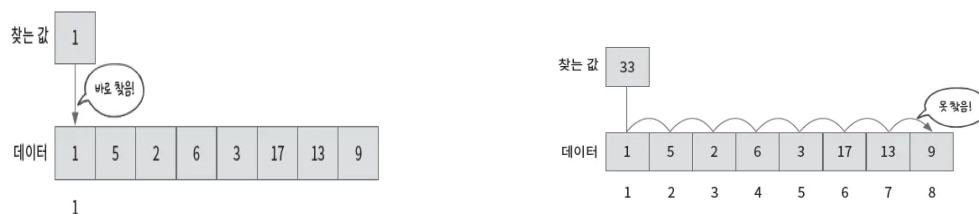
알고리즘의 성능을 나타내는 지표로, 입력 크기에 대한 연산 횟수의 상한을 의미
(시간 복잡도는 낮을 수록 좋음)

- 1차원 배열 검색하기

1차원 배열에 값이 있을 때 특정 값을 배열의 맨 앞에서 순서대로 검색한다고
해볼 때 값을 가장 빨리 찾는 경우와 가장 늦게 찾는 경우는 언제인가?

가장 빨리 찾는 방법: 검색시작 위치에 찾을 값이 위치한 경우

가장 늦게 찾는 방법: 값이 없거나 가장 마지막에 위치한 경우



따라서 알고리즘의 수행 시간 측정 방법은 절대 시간 측정과 시간 복잡도를 측정하는 방법
이 있음

- 절대 시간 측정 방법

절대 시간 측정 방법: 배열에서 검색하는 프로그램을 작성한 다음 프로그램을 실행하여 결
과가 나올 때까지의 시간을 측정.

단점) 프로그램을 실행하는 환경에 따라 달라질 수 있음

- 시간 복잡도 측정 방법

시간 복잡도 측정 방법: 알고리즘이 시작한 순간부터 결과값이 나올때까지의 연산횟수 측정.
주의) 특정 입력 크기에 한하여 연산 횟수를 기준으로 시간 복잡도 측정하면 안됨.

(*예를 들어 앞서 사용한 배열을 맨 앞부터 하나씩 검사하는 알고리즘은 최선의 경
우 연산횟수는 1번 최악의 경우 8번이 됨.)

점근적 표기법: 입력 크기에 따른 연산횟수의 추이를 활용해서 시간복잡도를 표현하는 방법

- 빅오 표기법: 입력 크기가 커질 때 알고리즘의 실행 시간이나 메모리 사용량이 얼마나 늘

어디까지 나타내는 방법(대표적인 점근적 상한법, 어떤 프로그램의 연산 횟수가 $f(x)$ 라고 할 때 함수의 최고차항을 남기고 차수를 지워 $O(\dots)$ 와 같이 표기)

점근적 상한선: 알고리즘이 최악의 상황일 때 차지하는 최대 성능

수식	빅오 표기	설명
$3x^2 + 5x + 6$	$O(x^2)$	다항함수로 구성되어 있으므로 최고차항 x^2 만 남습니다.
$x + \log x$	$O(x)$	다항함수와 로그함수로 구성되어 있으므로 증가폭이 더 낮은 로그함수는 사라지고 다항함수만 남습니다.
$2^x + 10x^5 + 5x^2$	$O(2^x)$	지수함수는 다항함수보다 빠르게 증가하므로 지수함수만 남습니다.
$5x^2 - 6x$	$O(x^2)$	최고차항 x^2 만 남습니다.

why?

ex)

```
def solution(n):
    count = 0
    # 반복문 1 : n^2번 연산 수행
    for i in range(n):
        for j in range(n):
            count += 1
    # 반복문 2 : n번 연산 수행
    for k in range(n):
        count += 1
    # 반복문 3 : 2n번 연산 수행
    for i in range(2*n):
        count += 1
    # 반복문 4 : 5번 연산 수행
    for i in range(5):
        count += 1

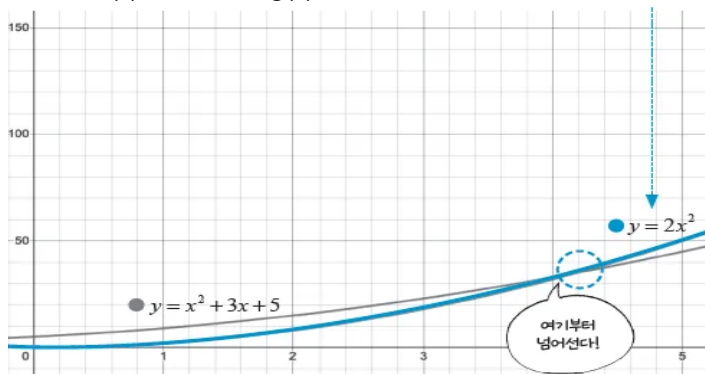
    print(count) # 59(n이 6일 때, 6^2 + 6 + 2*6 + 5 = 59)

solution(6) # 함수 호출
```

`solution()` 함수는 식으로 $f(x)=x^2+3x+5$ 로 나타낼 수 있음
 이때 다음을 만족하는 C 가 있으면 $f(x)$ 의 최악의 시간 복잡도는 $O(g(x))$

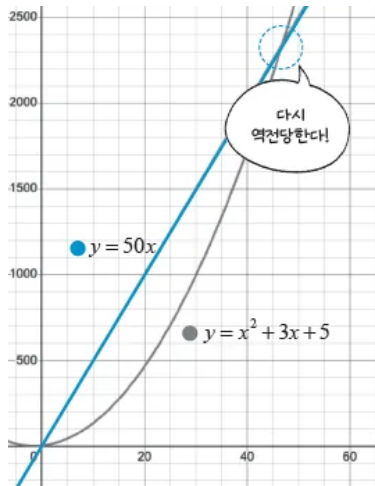
- 특정 x 시점 이후부터 항상 $f(x) \leq C * g(x)$ 를 만족
- C 는 상수

따라서 $f(x)$ 의 C 는 2, $g(x)$ 는 x^2



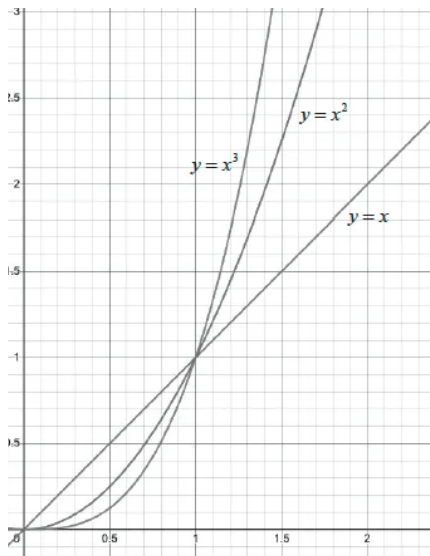
단, 위 공식을 만족하는 $g(x)$ 는 하나만 있지 않음

다만 이는 다시 역전 당하기 때문에 시간 복잡도는 $O(x^2)$



빅오 표기법에서 최고차항만 남기는 이유:

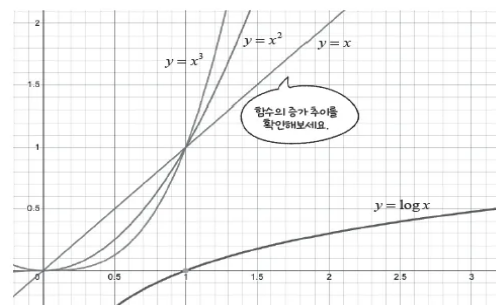
- $f(x)=x^2+3x+5$ 에서 최고 차항인 x^2 만 남김
- x^2 는 앞의 차수가 1이므로 제거할 것이 없음
- 시간 복잡도는 $O(x^2)$



데이터가 커질수록 세 그래프의 차이는 확연히 벌어짐.
 -> x^3 에 비해 x 와 x^2 는 무시할 수 있을 정도

로그함수는 다항함수보다 느리게 증가하고, 지수함수는 다항함수보다 빠르게 증가

함수 종류	예
지수함수	2^x
다항함수	$3x^2$
로그함수	$\log x$



- 시간 복잡도 활용 방법

문제에는 제한 시간이 있으므로 문제를 분석한 후에 빅오 표기법을 활용해서 해당 알고리즘을 적용했을 때 제한 시간 내에 출력값이 나올 수 있을지 확인할 수 있음

시간 복잡도	최대 연산 횟수
$O(N!)$	10
$O(2^N)$	20~25
$O(N^3)$	200~300
$O(N^2)$	3,000~5,000
$O(N\log N)$	100만
$O(N)$	1,000~2,000만
$O(\log N)$	10억

제한 시간이 1초인 문제에 각 시간 복잡도별로 허용할 수 있는 최대 연산 횟수



이 방식은 복잡도가 $O(N)$

연산 횟수가 1000만 이므로 데이터의 개수가 1000만 개 이하라면 사용 가능

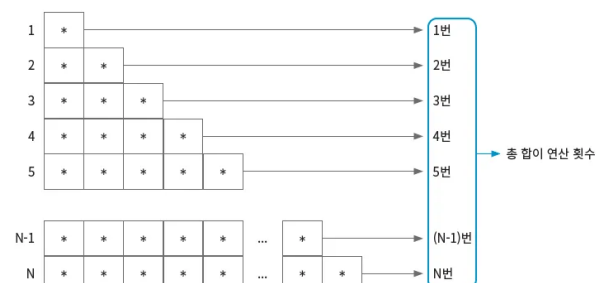
간 복잡도를 활용하여 불필요한 알고리즘을 제외하면 됨.

- 실전 활용

별 찍기 문제

N=3	N=5
*	*
**	**
***	***

연산 횟수 구하기->



빅오 표기법: $O(N^2)$

$$f(N) = 1 + 2 + \dots + N = \frac{N(N+1)}{2}$$

1.2 공간복잡도(Space Complexity)

프로그램 실행과 완료에 얼마나 많은 공간(메모리)가 필요한지를 나타냄.

공간 복잡도: 보조공간(Auxiliary Space)과 입력 공간(input size)을 합친 포괄적인 개념
보조 공간(Auxiliary Space): 알고리즘이 실행되는 동안 사용하는 임시 공간

표기법: 시간 복잡도와 유사하게 빅오 표기법 사용

- 고정 공간 (알고리즘과 무관한 공간): 코드 저장 공간, 단순 변수 및 상수
- 가변 공간 (알고리즘 실행과 관련있는 공간): 실행 중 동적으로 필요한 공간
- $S(P) = c + Sp(n)$
- c : 고정 공간
- $Sp(n)$: 가변 공간

고정 공간은 상수이므로 공간 복잡도는 가변 공간에 좌우된다.

ex)

```
int sum(int a[], int n)
{
    int x = 0;
    for(int i = 0; i < n; i++) {
        x = x + a[i];
    }
    return(x);
}
```

- int arr[n] : 4*n byte (입력 공간)
- int n : 4 byte (입력 공간)
- int x : 4 byte (보조 공간)
- int i : 4 byte (보조 공간)

4n+12 에 메모리를 요구

공간 복잡도 $O(N)$: 메모리가 입력 값에 따라 선형적으로 증가하기 때문

ex)

```
public int factorial(int n) {

    if(n == 1) {
        return n;
    }

    return n*factorial(n-1);
}
```

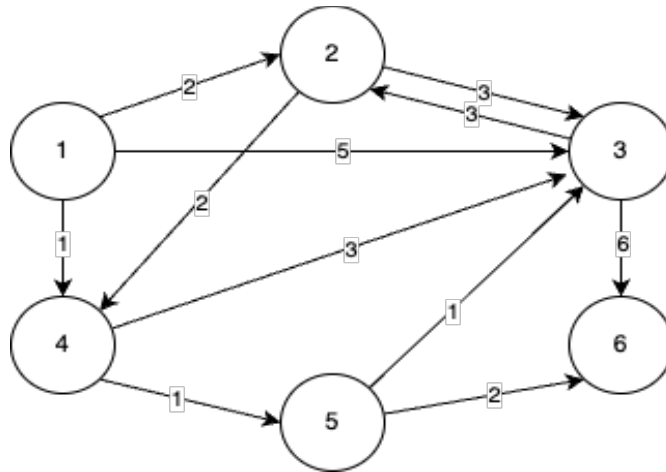
- 재귀 함수를 통해서 구현하므로 변수 n 에 따라서 변수 n 이 n 개가 만들어지게 됨
- factorial 함수를 재귀 함수로 1까지 호출하였을 경우, n 부터 1까지 스택에 쌓이게 됨
- 따라서 공간 복잡도는 $O(n)$

2.다익스트라 알고리즘

-다익스트라 알고리즘이란 유향 또는 무향 가중치 그래프에 대해 하나의 시작점에서 모든 정점까지의 최단 거리 및 최단 경로를 찾는 알고리즘. (에츠허르 다익스트라가 1956년에 발견해 1959년에 발표, 보통 모든 간선들의 가중치는 양수여야 함을 가정)

-알고리즘 원리

1. 출발 노드를 설정한다.
2. 최단 거리 테이블을 초기화한다.
3. 방문하지 않은 노드 중에서 최단 거리가 짧은 노드를 선택한다.
4. 해당 노드를 거쳐 다른 노드로 가는 비용을 계산하여 최단 거리 테이블을 갱신한다.
5. 위 과정에서 3번과 4번을 반복한다.



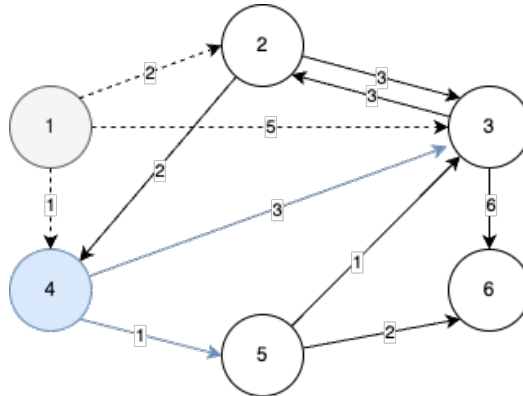
STEP 0

- 먼저 방문하지 않은 노드 중에서 최단거리가 짧은 노드를 선택해야 한다.
- 출발 노드에서 출발 노드로의 거리는 0으로 보기 때문에 처음에는 출발노드가 선택된다

노드번호	1	2	3	4	5	6
거리	0	무한	무한	무한	무한	무한

STEP 1

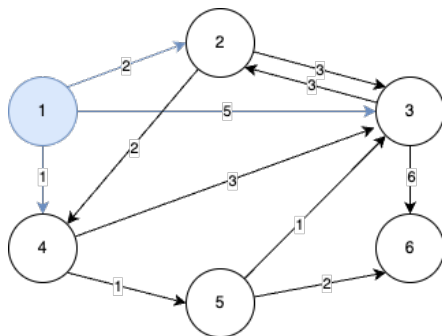
- 이제 1번 노드를 거쳐 다른 노드로 가는 비용을 계산
- 1번 노드와 연결된 모든 간선을 하나씩 확인
- 현재 1번 노드까지 오는 비용은 0이므로, 1번 노드를 거쳐서 2번, 3번, 4번 노드로 가는 최소비용은 차례대로 $2(0+2)$, $5(0+5)$, $1(0+1)$
- 무한으로 설정되어 있는 2, 3, 4번 노드로 가는 비용을 각각 새로운 값으로 갱신
- 현재 처리 중인 노드와 간선은 하늘색, 이미 처리한 노드는 회색, 이미 처리한 간선은 점선



노드번호	1	2	3	4	5	6
거리	0	2	5	1	무한	무한

STEP 2

- 이후 단계에서도 마찬가지로 방문하지 않은 노드 중에서 최단 거리가 가장 짧은 노드를 선택
- STEP 2에서는 4번 노드가 선택
- 4번 노드를 거쳐서 갈 수 있는 노드를 확인
- 4번 노드에서 갈 수 있는 노드는 3번과 5번
- 4번 노드까지의 최단 거리는 1이므로 4번 노드를 거쳐서 3번과 5번 노드로 가는 최소 비용은 차례대로 $4(1+3)$, $2(1+1)$
- 이 두 값은 기존 리스트에 담겨 있던 값보다 작으므로 리스트가 갱신



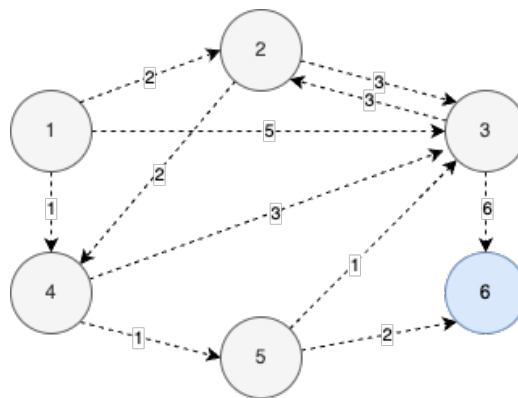
노드번호	1	2	3	4	5	6
거리	0	2	4	1	2	무한

이 과정들을 몇 번 반복하면 최단 거리 테이블이 나온다.

노드번호	1	2	3	4	5	6
거리	0	2	4	1	2	무한

노드번호	1	2	3	4	5	6
거리	0	2	3	1	2	4

노드번호	1	2	3	4	5	6
거리	0	2	3	1	2	4



노드번호	1	2	3	4	5	6
거리	0	2	3	1	2	4

<최종 거리 테이블>

- 최단 거리 테이블이 의미하는 바는 1번 노드로부터 출발 했을 때 2번, 3번, 4번, 5번, 6번 노드까지 가기 위한 최단 경로가 각각 2, 3, 1, 2, 4 라는 의미
- 방문하지 않은 노드 중에서 가장 최단 거리가 짧은 노드를 선택하는 과정을 반복
- 선택된 노드는 최단 거리가 완전히 선택된 노드이므로, 더이상 알고리즘을 반복해도 최단 거리가 줄어들지 않음
- STEP 2이후에 과정이 진행되었으나, 4번 노드에 대한 최단 거리는 더 이상 감소하지 않음
- 한 단계당 하나의 노드에 대한 최단 거리를 확실히 찾는 것

- A* 알고리즘

기본적으로는 다익스트라 알고리즘의 아이디어를 기반으로 하고 있지만, 여기에 휴리스틱 (Heuristic)이라는 개념을 더해서 성능을 비약적으로 높인 방식

기본적인 흐름

- 지금까지 온 거리(G)
- 앞으로 남은 거리(H)
- 총 예상 비용($F = G + H$)이 가장 낮은 후보부터 탐색

1. G는 시작점에서 현재 위치까지 실제로 이동한 누적 거리
(->한 칸 이동할 때 비용이 1이라면, 시작점에서 5칸 이동한 위치의 G값은 5)

2. H는 현재 위치에서 목적지까지의 예상 거리(추정값)

맨해튼 거리(Manhattan Distance): 목적지까지 남은 가로 거리 + 세로 거리

유클리드 거리(Euclidean Distance): 목적지까지의 대각선을 포함한 최소 직선 거리

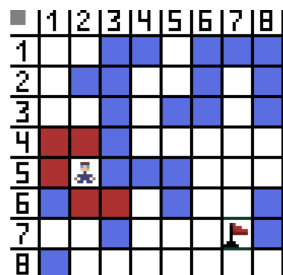
3. F 최종 비용: $H + G$

오픈리스트: 출발한 주인공 기준에서 인지하게 된 길 후보들
(미로에 어떤 길들이 있는지 전혀 모르기 때문에, 출발하는 플레이어 기준에서 갈 수 있다고 판단되는 모든 길들은 우선 오픈 리스트에 담아둠.)

클로즈 리스트: 클로즈 리스트는 탐색 완료한, 방문한 길

A* 알고리즘의 순서는 아래와 같다.

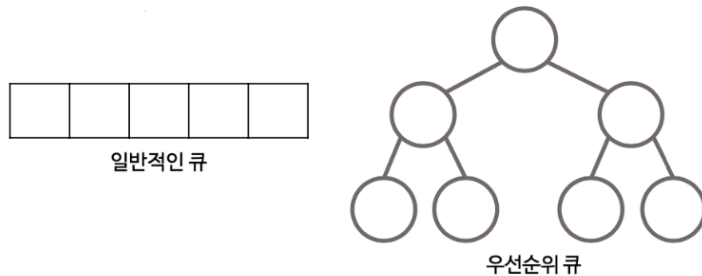
1. 시작 지점을 오픈 리스트에 추가한다.
2. 오픈 리스트에서 F값이 가장 작은 노드를 꺼낸다.
3. 이 노드를 클로즈 리스트에 넣는다.
4. 꺼낸 노드의 인접한 노드들을 확인한다.
 - 1.벽이거나 이미 닫힌 리스트에 있는 경우는 제외
 - 2.아직 발견하지 않았거나, 더 좋은 경로가 있다면 오픈 리스트에 넣는다.
5. 목적지에 도착할 때까지 위 과정을 반복한다.



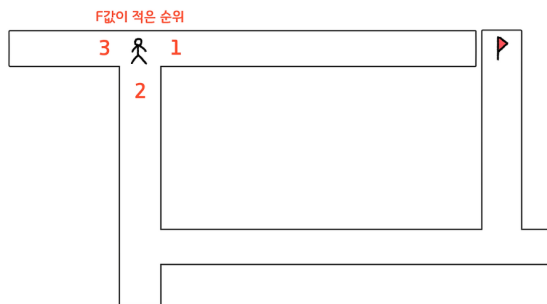
F값이 가장 작은 노드를 꺼내는 것->정렬을 유지하며 이진 탐색->우선순위 큐

- 우선순위 큐

이진 탐색 트리, 혹은 힙 구조로 이루어진 자료구조



우선 순위 큐는 포화 이진 트리의 모양을 유지하며, 최대 힙이나, 최소 힙이냐에 따라 루트 노드에 항상 최댓값 혹은 최솟값을 유지

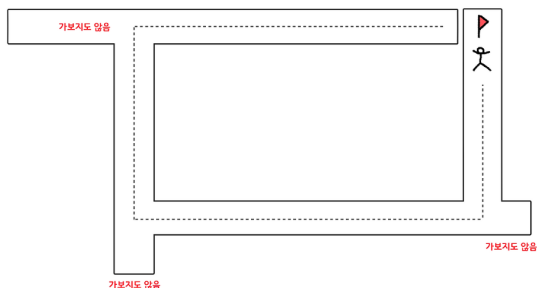


플레이어 기준에서 오른쪽으로 가는 길은 실제로 목적지까진 도달하지 못하는 막다른 길

A* 알고리즘은 우선 오른쪽 길로 가게 됨.

길이 F값이 가장 작은, 현재 기준에서 가장 현명해보이는 길이기 때문

단, 막다른 길이기 때문에 이 때 시점에서 오픈 리스트에서 가장 F값이 적은 값은, 1번으로 적어둔 길



순서대로 가다보면, 최대한 효율적인 길을 따라가다보니까 누가봐도 목적지까지 가지 않는 비효율적인 길은 우선 오픈 리스트에 넣어 고려는 해주지만 결국 가보지도 않게 됨 .

목적지까지 가는 더 효율적으로 보이는 길을 우선 시도해보다가, 결국 목적지에 도착하기 때문에 나머지 후보는 고려할 조차 없기 때문

만약 BFS나 다익스트라였다면, 저 가보지도 않았던 길들 역시 출발점으로부터 얼마큼의 최소 비용이 들었는지 모두 계산했을 것

A* 알고리즘은 이를 고려하지 않았고, 그만큼 시간적으로 이점을 봄